

Phsysics4LLM reproducibility study

Syntetic data & Canon layer ablations

Kirill Zemlianskii

March 2026

Transformer variants keep multiplying:

- Attention: sliding-window, MLA (DeepSeek), gated attention, DSA
- Positional encoding: RoPE, NoPE, ALiBi
- Gating: gated MLP, SwiGLU, MoE

Why can't we just compare them?

- Each change requires full re-training from scratch
- Academic scale (1.3B / 100B tokens): seed variance $\geq 2\text{--}4\%$
swamps architecture differences of $1\text{--}2\%$
- Result: architecture ablations are slow, noisy, and expensive

Idea: Decompose “intelligence” into atomic skills; design one controlled task per skill.

Advantages:

- Infinite high-quality data, zero cost
- Controlled difficulty — no skill mixing
- Short context, no CoT leakage
- Cheap enough to run at small scale

Design criterion: mental reasoning only, no CoT, short contexts, real-world relevant.

Physics 4.1 Task Suite

Depo	Reasoning depth
Brevo	Reasoning breadth
Capo	Knowledge capacity
Mano	Knowledge manipulation
Lano	Language structure

Setup: k -hop traversal over a directed permutation graph. No Chain-of-Thought allowed. Each node label can span multiple tokens.

Format:

$\langle \text{bos} \rangle \ x_1 \ y_1 \ x_2 \ y_2 \ \cdots \ \langle \text{query_}k \rangle \ q \rightarrow a$

Edges presented in *random* order. Model must chain k retrieval steps entirely in activations.

Why it's hard:

- max k determines the difficulty of the task
- No intermediate output tokens — all reasoning happens in the forward pass
- Must resolve depth- k pointer chains in a single pass

Setup: Recursive DAG traversal — given root q , output all reachable nodes in topological order. Node labels can span multiple tokens.

- Parallel fan-out: multiple reasoning threads must be held simultaneously
- Model must compute the full topological sort *before* emitting the first output token
- Graph size: up to 110 nodes

Motivation: Standard attention uses the *full* global mechanism just to copy information to a neighbouring token.

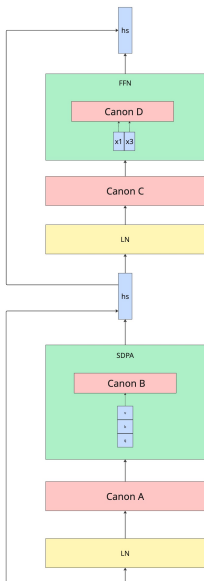
Solution: Lightweight horizontal residual links across neighbouring tokens:

$$h'_t = h_t + w_0 \odot h_t + w_1 \odot h_{t-1} + w_2 \odot h_{t-2} + w_3 \odot h_{t-3}$$

Element-wise, trainable weights per layer.

Implementation:

- Causal conv1d with kernel size 4 + explicit residual
- < 0.01% extra parameters for a 1.3B model



4 insertion points per transformer block:

- A** Pre-attention — before the attention sublayer
- B** Q/K/V projections — inside attention, applied to query, key, and value
- C** Pre-MLP — between attention output and the feed-forward sublayer
- D** Inside MLP — between the two linear projections

Synthetic tasks (RoPE + Canon-ABCD):

- Reasoning depth: $2\text{--}4\times$ improvement
- Reasoning breadth (Brevo): $+30\%$
- Knowledge manipulation (Mano): $+30\%$

Architecture comparisons:

- NoPE + Canon $\approx$ RoPE + Canon (length generalisation)
- GLA + Canon $\approx$ Mamba2 / GDN (simple baseline matches complex modern architectures)
- Real-world 1.3B / 100B tokens: Canon markedly improves NoPE and GLA

$\Rightarrow$ Synthetic task trends **mirror** real-world results.

- Can synthetic tasks be used to study training dynamics and the individual contribution of each architectural modification —
- Why does Canon help so much? Is it genuinely horizontal information mixing, or something else — like an implicit normalisation effect?
- How Canon parameters impact performance?
 - **Initialization** — does the starting magnitude of the conv weights matter?
 - **Kernel size** — how many past tokens should Canon look at?
 - **Placement** — which layer depth and insertion point (A/B/C/D) drives the gain?

Framework: Meta Lingua
(lingua_modified)

Model: Llama (RoPE), two
sizes:

- 8 layers $\times$ 512 dim, 8 heads
- 12 layers $\times$ 768 dim, 12
heads

Tied input/output embeddings,
bf16.

Training: 100 000 steps,
AdamW, lr 3×10^{-4} , warmup
1 000 — single A100

Tasks & tokenization:

- Depo / Brevo:
pass-through tokenizer,
batch 256, seq len 768
- Text: bytes tokenizer,
batch 128, seq len 2048

Loss: cross-entropy
(only on answer tokens for
synthetic tasks)

- F1 Training is noisy.** Seed variance dominates convergence speed; use final accuracy over multiple seeds.
- F2 Default init is not optimal.** $w_0=1/k$ beats zero, default random, and baseline; wrong init recovers slowly.
- F3 First canon layer captures most of the benefit.** Last-layer Canon baseline; first-layer Canon full Canon-ABCD.
- F4 Consistent improvement with along with any norm placement.** Pre-LN has the fastest convergence and biggest boost from canon.
- F5 Larger kernel size improves convergence.** $ks=6 > 4 > 2 > \text{baseline} > ks=1$; kernel of 1 is worse than no Canon.
- F6 Canon mixes information *and* normalises implicitly.** Higher weights on farther tokens; smoother gradients; fewer outlier activations.

- Training dynamics variance on synthetic data is determined by the task. What makes a task more noisy? How to design a task with training stability in mind?
- Is there a norm variation with training dynamics identical to canon.
- Reproduction with proper tokenizer.

Thank you!